

1. Comprendre la Programmation Orientée Objet

1.1. Définition de la POO

La **POO**, ou **Programmation Orientée Objet**, est une manière de programmer qui consiste à organiser le code autour d'**objets**.

Un **objet** représente quelque chose du monde réel ou logique :

- un utilisateur ;
- une voiture ;
- un produit ;
- un compte bancaire ;
- un bouton dans une application.

Chaque objet peut avoir :

1. Des données

Ce sont ses **caractéristiques**.

Exemple : une voiture peut avoir :

marque

couleur

vitesse

2. Des actions

Ce sont ses **comportements**.

Exemple : une voiture peut :

démarrer

accélérer

freiner

Définition simple à retenir

La **POO** est une façon de programmer où l'on crée des **objets** qui regroupent des **données** et des **actions**.

Exemple mental

Objet : Voiture

Données :

- couleur
- marque
- vitesse

Actions :

- démarrer()
- accélérer()
- freiner()

Idée clé

Au lieu d'écrire un programme comme une longue liste d'instructions, la POO permet de construire un programme avec des **objets qui collaborent entre eux**.

1.2. Pourquoi la POO a été créée

La **POO** a été créée pour mieux gérer les programmes qui deviennent **grands, complexes et difficiles à maintenir**.

Avant, on écrivait souvent le code comme une longue suite d'instructions. Plus le projet grandissait, plus le code devenait difficile à comprendre, modifier et réutiliser.

Problème

Quand un programme devient grand :

le code se répète

les erreurs augmentent

les modifications deviennent difficiles

l'organisation devient confuse

Solution apportée par la POO

La POO organise le programme en **objets**.

Chaque objet regroupe :

ses données

ses actions

ses responsabilités

Exemple simple

Au lieu d'avoir du code mélangé partout pour gérer un utilisateur, on crée un objet :

Utilisateur

- nom

- email

- seConnecter()

- seDéconnecter()

Idée clé à retenir

La **POO a été créée pour rendre le code plus organisé, plus réutilisable et plus facile à maintenir.**

1.3. Problèmes que la POO cherche à résoudre

La **POO** cherche à résoudre les problèmes qui apparaissent quand un programme devient **grand** et **difficile à gérer**.

Problèmes principaux

1. Code désorganisé

Quand tout est mélangé, on ne sait plus facilement où modifier le code.

logique utilisateur

logique produit

logique paiement

2. Répétition du code

Sans bonne organisation, on réécrit souvent les mêmes instructions à plusieurs endroits.

```
calculerPrix()
```

```
calculerPrix()
```

```
calculerPrix()
```

3. Maintenance difficile

Une petite modification peut casser plusieurs parties du programme.

4. Réutilisation limitée

Le code est parfois trop lié à un seul endroit, donc difficile à réutiliser ailleurs.

Solution apportée par la POO

La POO organise le code en **objets** avec des responsabilités claires.

Utilisateur

Produit

Commande

Paieement

Idée clé à retenir

La **POO aide à organiser, réutiliser et maintenir le code plus facilement.**

1.4. Différence entre programmation procédurale et programmation orientée objet

La **programmation procédurale** organise le code autour des **fonctions**.

La **programmation orientée objet** organise le code autour des **objets**.

1. Programmation procédurale

On écrit une suite d'instructions et de fonctions qui traitent des données.

Données → Fonctions → Résultat

Exemple :

nomUtilisateur

emailUtilisateur

connecterUtilisateur()

modifierEmail()

Ici, les **données** et les **fonctions** sont séparées.

2. Programmation orientée objet

On regroupe les données et les actions dans un même élément : **l'objet**.

Objet → Données + Actions

Exemple :

Utilisateur

- nom
- email
- seConnecter()
- modifierEmail()

Ici, les **données** et les **actions** sont ensemble.

Idée clé à retenir

La programmation procédurale pense en **fonctions**.

La programmation orientée objet pense en **objets avec responsabilités**.

1.5. Différence entre programmation fonctionnelle et programmation orientée objet

La **programmation fonctionnelle** organise le code autour des **fonctions**.

La **programmation orientée objet** organise le code autour des **objets**.

1. Programmation fonctionnelle

On utilise surtout des **fonctions** qui transforment des données.

Données → Fonction → Nouvelles données

Exemple :

```
calculerTotal(prix, quantité)
```

La fonction reçoit des données, fait un calcul, puis retourne un résultat.

2. Programmation orientée objet

On crée des **objets** qui possèdent leurs données et leurs actions.

Objet → Données + Actions

Exemple :

Produit

- prix
- quantité
- calculerTotal()

Ici, les données et l'action sont regroupées dans l'objet.

Idée clé à retenir

La programmation fonctionnelle pense en **transformations de données**.

La programmation orientée objet pense en **objets responsables de leurs données et actions**.

1.6. Quand utiliser la POO

On utilise la **POO** quand un programme contient plusieurs éléments qui ont des **données** et des **actions**.

Elle est surtout utile quand le projet devient **plus grand, plus structuré** ou **plus difficile à maintenir**.

À utiliser quand...

1. Le programme représente des éléments réels ou logiques

Exemples :

Utilisateur

Produit

Commande

Paielement

2. Chaque élément a ses propres données

Exemple :

Utilisateur

- nom
- email
- motDePasse

3. Chaque élément peut faire des actions

Exemple :

Utilisateur

- seConnecter()
- modifierProfil()
- seDéconnecter()

4. Le code doit être facile à modifier et réutiliser

La POO aide à organiser le code en blocs clairs et réutilisables.

Idée clé à retenir

On utilise la **POO** quand on veut construire un programme avec des **objets bien organisés**, ayant chacun leurs **données**, leurs **actions** et leurs **responsabilités**.

1.7. Quand éviter la POO

Il faut éviter la **POO** quand elle rend le programme **plus compliqué que nécessaire**.

La POO est utile pour organiser des projets complexes, mais elle n'est pas obligatoire partout.

À éviter quand...

1. Le programme est très simple

Exemple :

`additionner deux nombres`

`convertir une température`

`afficher un message`

Créer des classes pour cela serait inutile.

2. Le code est surtout une suite de calculs

Quand le programme transforme simplement des données, des fonctions peuvent suffire.

`données → calcul → résultat`

3. Les objets n'ont pas de vraie responsabilité

Mauvais exemple :

`ClasseCalcul`

`ClasseOutils`

`ClasseTraitement`

Si la classe ne représente rien de clair, elle ajoute de la confusion.

Idée clé à retenir

Il faut éviter la **POO** quand de simples **fonctions** rendent le code plus court, plus clair et plus facile à comprendre.

1.8. Avantages de la POO

La **POO** aide à créer un code plus **organisé**, plus **clair** et plus **facile à faire évoluer**.

Avantages principaux

1. Meilleure organisation

Le code est rangé dans des objets avec des responsabilités claires.

Utilisateur

Produit

Commande

Paielement

2. Réutilisation du code

Un objet ou une classe peut être utilisé à plusieurs endroits du programme.

Classe Utilisateur

→ utilisée dans plusieurs parties de l'application

3. Maintenance plus facile

Quand le code est bien séparé, on peut modifier une partie sans toucher à tout le programme.

4. Code plus proche du réel

La POO permet de représenter des éléments concrets ou logiques.

CompteBancaire

- solde
- déposer()
- retirer()

Idée clé à retenir

La **POO** permet de construire des programmes plus organisés, réutilisables et faciles à maintenir.

1.9. Limites de la POO

La **POO** est puissante, mais elle peut aussi rendre un programme **plus lourd** si elle est mal utilisée.

Limites principales

1. Trop de complexité

Créer beaucoup de classes pour un petit problème peut compliquer le code.

Additionner deux nombres

→ pas besoin d'une classe spéciale

2. Trop d'abstraction

Parfois, on crée des objets très théoriques, difficiles à comprendre.

GestionnaireDeGestionDeTraitement

Ce genre de classe rend le code moins clair.

3. Mauvais découpage

Si les responsabilités sont mal séparées, les objets deviennent confus.

Utilisateur

- seConnecter()
- payerCommande()
- envoyerEmail()
- générerFacture()

Ici, l'objet fait trop de choses.

4. Performance parfois moins directe

La POO peut ajouter plus de structure que nécessaire pour certains traitements simples ou très rapides.

Idée clé à retenir

La **POO** devient une limite quand elle ajoute plus de complexité qu'elle n'apporte de clarté.

1.10. Exemples concrets de pensée orientée objet

Penser en **POO**, c'est regarder un problème et se demander :

Quels sont les objets ?

Quelles sont leurs données ?

Quelles sont leurs actions ?

Exemple 1 : Utilisateur

Objet : Utilisateur

Données :

- nom
- email

Actions :

- seConnecter()
- modifierProfil()

Exemple 2 : Produit

Objet : Produit

Données :

- nom
- prix
- stock

Actions :

- afficherPrix()
- réduireStock()

Exemple 3 : Compte bancaire

Objet : CompteBancaire

Données :

- solde
- titulaire

Actions :

- déposer()
- retirer()

Idée clé à retenir

La pensée orientée objet consiste à découper un programme en **objets**, puis à donner à chaque objet ses **données**, ses **actions** et sa **responsabilité**.